MINISTERUL EDUCAȚIEI ȘI CERCETĂRII AL REPUBLICII MOLDOVA

Universitatea Tehnică a Moldovei

Facultatea Calculatoare, Informatică şi Microelectronică

Departamentul Inginerie Software și Automatică

RAPORT

Lucrare de laborator Nr. 7
Disciplina: ,,Tehnici de Programare”

Realizat: **St. gr. SI-264,**
Zlatin Nichita

Verificat: **Reițman Patricia,**
asist. Univ.

Chișinău 2026

Introducere

Scopul lucrării: Stăpânirea utilizării colecțiilor din Python (`set`, `list`, `dict`) și a sintaxelor de tip *comprehensions* pentru procesarea, curățarea și analiza datelor din fișiere de sistem. Lucrarea urmărește dezvoltarea capacității de evaluare a riscurilor de securitate (*PrivEsc*), implementarea analizei diferențiale prin operații pe mulțimi pentru identificarea modificărilor neautorizate, serializarea datelor în format JSON, scrierea testelor unitare automatizate cu `pytest` și versionarea codului pe depozitul GitHub.

Sarcina lucrării: Sarcina constă în dezvoltarea modulului de enumerare `enum.py` pentru filtrarea datelor și evaluarea riscurilor SUID, Sudo și Cron, urmată de salvarea raportului în `enum.json`. De asemenea, include realizarea analizei diferențiale în `diferenta.py` pentru identificarea executabilelor SUID noi și periculoase prin operații cu mulțimi, scrierea testului unitar `test_diferenta.py` rulat cu `pytest` și încărcarea livrabilelor pe depozitul GitHub `TP-Proiect-ZlatinNichita-SI-264`.

Mersul Lucrarii:

Partea 1: **LUCRAREA A (nucleu). enum.py: adună și grupează**

```python
# A1

import argparse, json, os

def citeste_linii(cale):  4 usages  Zlatin Nichita
    result = []
    with open(cale, encoding="utf-8") as f:
        # TODO: intoarce lista liniilor .strip(), fara cele goale si fara cele cu #
        for i in f:
            linie_curata = i.strip()
            if linie_curata and not linie_curata.startswith("#"):
                result.append(linie_curata)

    return result
```

Figura 1.1: **A1. Scheletul și prima listă**

```python
# A2

def nume(cale :{rsplit} ):  4 usages  Zlatin Nichita
    # TODO: intoarce ce e dupa ultimul "/"  (vezi rsplit)
    return cale.rsplit('/', 1)[-1]
```

Figura 1.2: **A2. Nume unice: de la listă la mulțime**

```python
# A3

GTFO = {"find", "vim.basic", "vim", "nmap", "cp", "bash",
            "less", "awk", "python3", "perl"}

def suid_constatari(cale):  1 usage  Zlatin Nichita
    result = []
    cai = citeste_linii(cale)
    # TODO: pentru fiecare cale UNICA, in ordine sortata, un dict:
    #   {"cale": c, "nume": nume(c),
    #    "risc": "mare" daca nume(c) in GTFO, altfel "normal"}
    for i in sorted(set(cai)):
        cale = i
        nume_cale = nume(cale)
        if nume_cale in GTFO:
            risc = "mare"
        else:
            risc = "normal"
        result.append({"cale": cale, "nume": nume_cale, "risc": risc})

    return result
```

Figura 1.3: **A3. Constatările SUID: listă de dicționare**

```python
# A4

def sudo_constatari(cale):  1 usage  Zlatin Nichita
    out = []
    for ln in citeste_linii(cale):
        user, drept = ln.split(" : ", 1)  # despachetare in tuplu
        # TODO: adauga {"user": user.strip(), "drept": drept.strip(),
        #   "risc": "mare" daca "NOPASSWD" in drept, altfel "normal"}
        if "NOPASSWD" in drept:
            risc = "mare"
        else:
            risc = "normal"
        out.append({"user": user.strip(), "drept": drept.strip(), "risc": risc})

    return out

def cron_constatari(cale):  1 usage  Zlatin Nichita
    # TODO: campuri = ln.split(); user = campuri[5];
    #   comanda = " ".join(campuri[6:]); dict cu user si comanda
    out = []
    for ln in citeste_linii(cale):
        campuri = ln.split()
        user = campuri[5]
        commanda = " ".join(campuri[6:])
        out.append({"user": user.strip(), "commanda": commanda})

    return out
```

Figura 1.4: **A4. sudo și cron: tuplul și gruparea**

```python
def enumereaza(d):  1 usage  Zlatin Nichita
    return {
        "suid": suid_constatari(os.path.join(d, "suid.txt")),
        "sudo": sudo_constatari(os.path.join(d, "sudo.txt")),
        "cron": cron_constatari(os.path.join(d, "crontab.txt")),
    }
```

Figura 1.5: **A4. sudo și cron: tuplul și gruparea (Continuare)**

```python
# A5

def main():  1 usage  Zlatin Nichita
    p = argparse.ArgumentParser()
    p.add_argument("capturi")
    p.add_argument( *name_or_flags: "--json", action="store_true")
    a = p.parse_args()
    raport = enumereaza(a.capturi)
    sudo_users = {u['user'] for u in raport["sudo"]}
    cron_users = {u['user'] for u in raport["cron"]}
    all_users = sudo_users | cron_users
    if a.json:
        print(json.dumps(raport, indent=2, ensure_ascii=False)); return
    for categorie, lista in raport.items():
        mari = [c for c in lista if c.get("risc") == "mare"]
        print(f"{categorie}: {len(lista)} gasite, {len(mari)} cu risc mare")
    # TODO: utilizatorii implicati = multimea userilor din sudo REUNITA cu cei din cron
    print(f"utilizatorii implicati: {", ".join(sorted(all_users))}")
    with open("enum.json", "w", encoding="utf-8") as f:
        json.dump(raport, f, indent=2, ensure_ascii=False)
    print("Raport salvat in enum.json")


if __name__ == "__main__":
    main()
```

Figura 1.6: **A5. Raportul: JSON și rezumat pe categorii**

Partea 2: **LUCRAREA B (extindere). Ce e nou față de un system curat**

```python
from arsenal.privesc.enum import citeste_linii, nume, GTFO

# B1

def cai(fisier):  2 usages  Zlatin Nichita
    # TODO: intoarce MULTIMEA cailor din fisier (foloseste citeste_linii din enum)
    return set(citeste_linii(fisier))


baza = cai("arsenal/privesc/start/baza_suid.txt")
acum = cai("arsenal/privesc/start/suid.txt")
print("baza:", len(baza), " acum:", len(acum))
```

Figura 2.1: **B1. Două mulțimi: curat și acum**

```python
# B2

noi = acum - baza
print(len(noi), "SUID noi fata de sistemul curat")
for c in sorted(noi):
    print(" ", c)
```

Figura 2.2: **B2. Diferența: ce a apărut**

```python
# B3

risc = {nume(c): ("mare" if nume(c) in GTFO else "normal") for c in noi}
periculoase = [c for c in sorted(noi) if nume(c) in GTFO]
print("periculoase NOI:", periculoase)
```

Figura 2.3: **B3. Dicționar de risc și filtrul final**

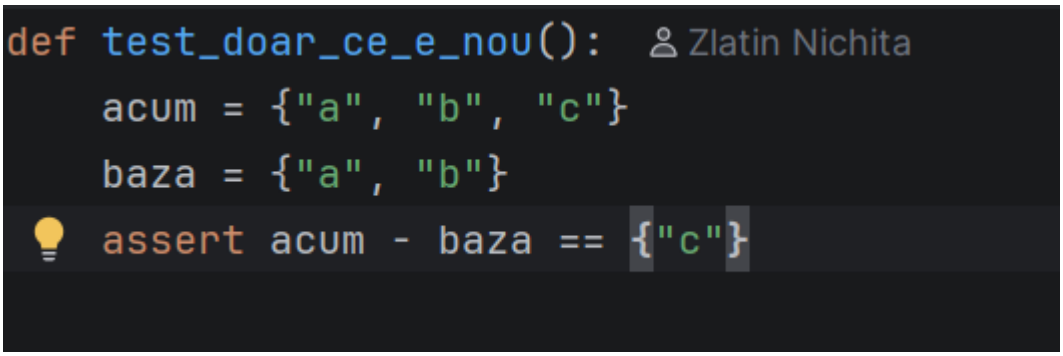


Figura 2.4: **B4. Un test care prinde regresia**

Concluzia lucrării: În urma realizării acestei lucrări de laborator, s-au aplicat cu succes colecțiile din Python (`set`, `dict`, `list`) pentru enumerarea configurărilor de sistem și identificarea rapidă a potențialelor vulnerabilități de escaladare a privilegiilor. S-a automatizat structurarea datelor în format JSON prin modulul `enum.py` și s-a implementat analiza diferențială în `diferenta.py` pentru detectarea executabilelor noi cu risc ridicat. Corectitudinea logicii a fost validată prin teste automatizate cu `pytest`, iar livrabilele finale au fost sincronizate cu succes pe depozitul GitHub `TP-Proiect-ZlatinNichita-SI-264`.